

傅里叶变换 Flutter 应用开发

包含符号推导与数值可视化，并由规则优先的 Python 后端提供支持

中文检查版

以 Flutter 应用作为项目主体

后端仅作为符号傅里叶推导的支持服务

目录

1. 摘要

2. 引言

3. 背景与设计理由

- 傅里叶变换背景
- 项目动机
- 项目目标
- 范围与限制

4. 整体系统架构

- 系统概述
- Flutter 前端
- FastAPI 符号后端
- 前后端通信

5. Flutter 应用设计

- 用户界面
- 公式输入
- 示例导航
- LaTeX 渲染

6. 符号傅里叶变换引擎

- 基于定义的框架
- 规则匹配策略
- 基于分布理论的变换
- 通用变换族
- 错误处理

7. 实现细节

- 后端命令与函数
- 关键代码片段
- Matcher 设计
- API 设计
- 数据流

8. 测试与评估

- 后端回归测试
- Flutter 测试
- 代表性测试用例
- 性能观察

9. 讨论

- 当前能力
- 支持的函数类别
- 限制
- 与通用 CAS 系统的比较

10. 结论与未来工作

参考文献

附录 A. 问答摘要

附录 B. 额外代码片段

附录 C. 示例输入与输出

摘要

本文介绍一个面向智能设备的傅里叶变换 Flutter 学习应用。原始项目要求强调 Flutter 编程、模拟器或真机测试、逐步推导，以及尽可能通过图像或示意图帮助理解傅里叶变换。因此，本文将 Flutter 应用作为主要用户界面和项目主体。

开发过程中曾考虑将符号处理完全放在 Flutter/Dart 本地完成。Flutter 和 Dart 适合交互、局部数值实验和视觉展示，但对于涉及 Dirac delta、Heaviside 阶跃函数、Cauchy principal value、工程风格 LaTeX 步骤等内容的可靠符号傅里叶推导并不充分。因此，最终设计引入 Python FastAPI 后端作为支持性的符号计算服务。

最终系统结合了 Flutter 学习界面、结构化示例、LaTeX 显示、数值可视化，以及规则优先的符号推导后端。目前后端测试结果为 68 passed，Flutter 测试结果为 6 passed，另有 1 个可选前后端 e2e 测试默认跳过。

引言

本项目原始定义是开发用于傅里叶变换的 Flutter app，并运行在智能设备上。应用需要在模拟器或实际设备上进行测试，显示逐步求解过程，并在合适位置提供图像、图表或说明性可视化。

因此，Flutter app 是本项目的中心，而后续引入的后端应理解为服务于符号推导的支持组件，而不是替代 Flutter 应用目标。

从教学角度看，学生学习傅里叶变换时不仅需要最终频域结果，还需要理解结果如何得到。因此，一个有用的学习应用应支持表达式输入、清晰公式、引导性示例、推导步骤和视觉直觉。

背景与设计理由

傅里叶变换背景

本项目采用工程傅里叶变换约定：

$$X(\omega) = \int x(t) \exp(-j \omega t) dt$$

显示层使用工程中的虚数单位 j ，并将 ω 视为实角频率。当普通积分不足以表达结果时，系统使用 delta、unit step、PV 和 sign 等分布理论对象。

项目动机

通用计算机代数系统可以计算很多符号变换，但输出形式并不总是适合工程教学。结果可能包含 Piecewise、RootSum、复杂条件或其他符号结构。这些结果虽然可能正确，但学生不容易与教材中的变换对联系起来。

```
CAS-style issue:
Piecewise(... complex conditions ...)

Engineering textbook target:
F{1/(t^2+1)} = pi * exp(-abs(omega))
```

项目目标

- 开发 Flutter 前端，用于表达式输入、示例、LaTeX 显示和图形化学习。
- 为支持的傅里叶变换问题提供逐步推导。
- 解释为什么仅靠 Flutter 本地符号处理无法满足最终推导目标。
- 使用 Python 后端支持分布理论相关的符号推导。
- 通过自动化测试验证后端数学输出和前端布局表现。

范围与限制

该应用不是无限制的计算机代数系统。它支持一个定义明确的工程范围，包括常数、冲激、阶跃、三角函数、指数函数、PV 倒数形式、多项式分布、多项式乘阶跃、部分有理函数、有限窗和支持项的线性组合。

整体系统架构

系统概述

本系统通过职责边界来解释，而不是依赖复杂流程图。Flutter 负责面向用户的学习任务，Python 后端负责符号推导。两层通过 JSON API 通信。

层	主要职责
Flutter 应用	智能设备界面、键盘输入、结构化示例、LaTeX 显示、本地数值可视化和小屏布局处理。
FastAPI 符号后端	表达式解析、规则优先符号推导、分布理论规则、result_latex 格式化

	和 <code>steps_latex</code> 生成。
通信边界	Flutter 将用户表达式发送到 <code>/fourier</code> ，并接收 <code>input_latex</code> 、 <code>result_latex</code> 和 <code>steps_latex</code> 。

Flutter 前端

前端负责输入、示例、公式预览、结果展示、数值可视化和响应式布局。由于原始项目要求明确关注 Flutter app，因此 Flutter 保持为主要应用层。

FastAPI 符号后端

后端是支持性的符号服务。引入后端的原因是可靠的符号傅里叶变换需要表达式解析、代数改写、分布理论规则和可控 LaTeX 输出。

前后端通信

前端会将实际用户输入发送到后端。这说明应用并不限于固定示例，而是可以在支持的规则范围内处理一般用户输入。

```
final res = await http.post(
  uri,
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({'expression': expression}),
);
```

Flutter 应用设计

用户界面

首页提供表达式输入、公式预览、示例导航、FFT 控制，以及进入符号和数值结果视图的入口。

公式输入

键盘和表达式处理支持常见信号表达式，例如 $u(t)$ 、 $\delta(t)$ 、 $\sin$ 、 $\cos$ 、 $\exp$ 、幂和分数。应用也保留本地数值处理能力，用于可视化。

示例导航

示例列表从简单变换对扩展为结构化教学示例。每个示例包含表达式、标题、类别、输入 LaTeX、变换对象和说明。

```
class FourierExample {
    final String expression;
    final String title;
    final String category;
    final String inputLatex;
    final String transformLatex;
    final String description;
}
```

LaTeX 渲染

前端使用 LaTeX 渲染公式预览、符号结果、推导步骤和公式参考。长公式和长说明通过局部滚动容器处理，从而避免小屏幕溢出。

```
class ScrollableMathLine extends StatefulWidget {
    final String latex;
    final TextStyle? textStyle;
    final String? semanticsLabel;
}
```

符号傅里叶变换引擎

基于定义的框架

每个教学推导都以傅里叶变换定义为基础。后端可以使用已知变换对和性质，但展示给用户的步骤应是数学解释，而不是内部 matcher 逻辑追踪。

规则匹配策略

后端先使用规则，再考虑不够可控的符号形式。这能让输出更可预测，并更接近工程教材记号。

```
rules = [
    exact_matches,
    known_transform_pairs,
    distribution_rules,
    property_based_rules,
    decomposition_rules,
    controlled_fallback,
]
```

基于分布理论的变换

分布理论相关变换是使用 Python 后端的重要原因。例如，多项式乘阶跃函数需要 delta 导数和 PV 项。

```
def _rule_poly_times_step_distribution(f):
    if not (isinstance(f, Mul) and f.has(Heaviside)):
        return None

    def _basis(n):
        if n == 0:
            return pi*DiracDelta(omega) - I*(1/omega)
        return (
            pi*(I**n)*Derivative(DiracDelta(omega), (omega, n))
            + ((-1)**n)*(I**(n-1))*factorial(n)*PV(1/(omega**(n+1)))
        )
```

通用变换族

引擎支持参数化的变换族，而不只是固定示例。支持范围包括移位冲激、移位阶跃、带频率和相位的正弦函数、复指数、单边指数、PV 倒数形式、多项式分布、多项式乘阶跃、部分有理函数、有限窗和线性组合。

错误处理

如果表达式超出支持范围，后端应返回受控 fallback 或错误，而不是给出不可靠的闭式结果。

实现细节

后端命令与函数

后端符号 API 通过 /fourier endpoint 实现。响应结构明确区分原始输入、最终结果和推导步骤。

```
class FourierResponse(BaseModel):
    ok: bool
    input_latex: str
    result_latex: str
    steps_latex: list[str]
    error: str | None = None
```

关键代码片段

以下函数用于整理面向教学展示的推导步骤，过滤内部实现细节和不希望展示的符号结构。

```
def _teaching_steps(f, X, steps):
    cleaned = []
    for raw in steps or []:
        s = _format_step_display_latex(str(raw).strip())
```



```
    if 'Method:' in s or '_rule_' in s or 'matcher' in s:
        continue
    if 'Piecewise' in s or 'RootSum' in s or 'meijerg' in s:
        continue
    cleaned.append(s)
return cleaned
```

Matcher 设计

Matcher 采用规则优先设计。也就是说，精确匹配和已知变换族会先于 fallback 策略执行。这样支持范围更明确，也更容易测试。

API 设计

API 有意保持简单：前端发送表达式字符串，后端返回可直接用于 LaTeX 展示的字段。这避免了向前端暴露后端内部符号对象。

数据流

数据流可以用正常行文描述：用户在 Flutter 中输入表达式，前端向 /fourier 发送 POST 请求，后端解析并匹配规则，生成 LaTeX 结果，最后由 Flutter 渲染。

测试与评估

后端回归测试

```
cd backend
.\.venv\Scripts\python.exe -m pytest tests -q
```

Result: 68 passed

后端测试覆盖严格 result_latex 检查、教学型 steps_latex 检查、代表性 step snapshot，以及对 Piecewise、RootSum、arg、polar_lift、meijerg、matcher、debug、srepr 等不良输出的检查。

Flutter 测试

```
cd flutter_app
flutter test
```

Result: 6 passed, 1 skipped

Flutter 测试覆盖响应式断点、首页渲染、示例导航、多种屏幕尺寸无布局异常、长公式和长说明的局部滚动，以及 FT Steps 页面窄屏长公式显示。

代表性测试用例

代表性后端 step cases 包括 1、 $\delta(t-3)$ 、 $u(t)$ 、 $t \cdot u(t)$ 、 $\sin(t) \cdot u(t)$ 、 $\frac{1}{3}t^{-2}$ 、 $\frac{1}{t^2+1}$ 、 $u(t-2)-u(t-5)$ 。这些用例同时检查数学结果质量和教学步骤清晰度。

性能观察

本地数值可视化保留在 Flutter 中，以保证交互响应；符号推导交给后端处理。这样的分离避免了交互图表受网络延迟影响，同时把精确符号工作放在更适合的 Python 环境中。

讨论

当前能力

应用可以在定义好的工程范围内支持一般用户输入。它并不只依赖固定示例，虽然示例列表可以帮助用户理解支持的表达式类别。

支持的函数类别

函数类别	示例
冲激与阶跃	$\delta(t)$, $\delta(t-a)$, $u(t)$, $u(t-a)$
三角与指数	$\sin(a \cdot t + b)$, $\cos(a \cdot t + b)$, $\exp(j \cdot \omega_0 \cdot t)$, $\exp(-a \cdot t)u(t)$
分布与多项式	$PV(1/t)$, $1/(a \cdot t + b)$, t^n , $t^n u(t)$
有理函数与窗函数	$1/(t^2 + a^2)$, $t/(t^2 + a^2)$, $u(t-a)-u(t-b)$

限制

系统不是无限制 CAS。超出支持范围的表达式可能返回积分形式或错误。这一限制是有意的，因为项目优先考虑可解释、可测试、工程风格的输出。

与通用 CAS 系统的比较

通用 CAS 更重视符号完整性。本项目更重视教学可读性、工程记号和逐步推理。规则优先方法在支持范围内可以给出更清晰的输出，但需要持续扩展规则。

结论与未来工作

本项目实现了一个基于 Flutter 的傅里叶变换学习应用，支持符号推导和数值可视化。项目仍然符合原始 Flutter 智能设备应用要求，而 Python 后端在 Flutter-only 处理不足时提供支持性的符号计算能力。

未来工作包括扩展规则覆盖范围、模块化后端、改进 unsupported case 提示、加入截图级视觉回归测试，以及支持导出课堂用推导报告。

参考文献

- Signals and Systems 课程材料与标准傅里叶变换表。
- Flutter 官方文档，用于应用开发和 widget 测试。
- FastAPI 官方文档，用于后端 API 实现。
- SymPy 官方文档，用于符号解析和代数处理。

附录 A. 问答摘要

详细 Q&A 文件保存在 docs/QA.md，邮件简洁版保存在 docs/QA_email_brief.md。这些文件回答了老师关于一般用户输入能力、一定范围内的通用傅里叶变换支持、PV 解释、示例、LaTeX 显示、推导步骤、测试和 dissertation 格式的问题。

附录 B. 额外代码片段

更长的实现细节，例如完整规则函数、本地数值 FFT 代码、parser 细节和部署配置，建议保留在源码仓库或附录中，而不是放在正文主体。

附录 C. 示例输入与输出

测试数据参考保存在 test.md，其中包含已经实现的变换用例，以及未来可继续扩展的 pending transform pairs。